

Requirements from the customer's perspective

Gerhard, a senior manager at Contoso Pharmaceuticals, was meeting with Cynthia, the manager of Contoso's IT department. "We need to build a chemical tracking information system," Gerhard began. "The system should keep track of all the chemical containers we already have in the stockroom and in laboratories. That way, the chemists can get some chemicals from someone down the hall instead of always buying a new container. This should save us a lot of money. Also, the Health and Safety Department needs to generate government reports on chemical usage and disposal with a lot less work than it takes them today. Can you build this system in time for the compliance audit in five months?"

"I see why this project is important, Gerhard," said Cynthia. "But before I can commit to a schedule, we'll need to understand the requirements for the chemical tracking system."

Gerhard was confused. "What do you mean? I just told you my requirements."

"Actually, you described some general business objectives for the project," Cynthia explained. "That doesn't give me enough information to know what software to build or how long it might take. I'd like to have one of our business analysts work with some users to understand their needs for the system."

"The chemists are busy people," Gerhard protested. "They don't have time to nail down every detail before you can start programming. Can't your people figure out what to build?"

Cynthia replied, "If we just make our best guess at what the users need to do with the system, we can't do a good job. We're software developers, not chemists. I've learned that if we don't take the time to understand the problem, nobody is happy with the results."

"We don't have time for all that," Gerhard insisted. "I gave you my requirements. Now just build the system, please. Keep me posted on your progress."

Conversations like this take place regularly in the software world. Customers who request a new system often don't understand the importance of obtaining input from actual users of the proposed system as well as other stakeholders. Marketers with a great product concept believe that they can adequately represent the interests of prospective buyers. However, there's no substitute for eliciting requirements directly from people who will actually use the product. Some agile development methods recommend that an on-site customer representative, sometimes called a product owner, work closely with the development team. As one book about agile development said, "The project is *steered* to success by the customer and programmers working in concert" (Jeffries, Anderson, and Hendrickson 2001).

Part of the requirements problem results from confusion over the different levels of requirements described in Chapter 1, “The essential software requirement”: business, user, and functional. Gerhard stated some business objectives, benefits that he expects Contoso to enjoy with the help of the new chemical tracking system. Business objectives are a core element of the business requirements. However, Gerhard can’t entirely describe the user requirements because he’s not an intended user of the system. Users, in turn, can describe tasks they must be able to perform with the system, but they can’t state all the functional requirements that developers must implement to let them accomplish those tasks. Business analysts need to collaborate with users to reach that deeper understanding.

This chapter addresses the customer-development relationship that is so critical to software project success. We propose a Requirements Bill of Rights for Software Customers and a corresponding Requirements Bill of Responsibilities for Software Customers. These lists underscore the importance of customer—and specifically end user—involvement in requirements development. This chapter also discusses the critical issue of reaching agreement on a set of requirements planned for a specific release or development iteration. Chapter 6, “Finding the voice of the user,” describes various types of customers and users and ways to engage appropriate user representatives in requirements elicitation.



Deliverable: Rejected

I heard a sad story when I visited a corporate IT department once. The developers had recently built a new information system for use within the company. They had obtained negligible user input from the beginning. The day the developers proudly unveiled their new system, the users rejected it as completely unacceptable. This came as a shock because the developers had worked hard to satisfy what they perceived to be the users’ needs. So what did they do then? They fixed it. Companies always fix the system when they get the requirements wrong, yet it always costs much more than if they had engaged user representatives from the outset.

The developers hadn’t planned to spend time fixing the flawed information system, of course, so the next project in the team’s queue had to wait. This is a lose-lose-lose situation. The developers were chagrined, the users were unhappy because their new system wasn’t available when they expected it, and the executives were upset over a lot of wasted money and the opportunity costs of delaying other projects. Extensive and ongoing customer engagement from the start could have prevented this unfortunate—but not uncommon—project outcome.

The expectation gap

Without adequate customer involvement, the inescapable outcome at the end of the project is an expectation gap, a gulf between what customers really need and what developers deliver based on what they heard at the beginning of the project (Wiegers 1996). This is shown as the dashed lines in Figure 2-1. As with the previous story, the expectation gap comes as a rude surprise to all stakeholders. In our experience, software surprises are never good news. Requirements also get out of date because of changes that occur in the business, so ongoing interactions with customers are vital.

The best way to minimize the expectation gap is to arrange frequent contact points with suitable customer representatives. These contact points can take the form of interviews, conversations, requirements reviews, user interface design walkthroughs, prototype evaluations, and—with agile development—user feedback on small increments of executable software. Each contact point affords an opportunity to close the expectation gap: what the developer builds is more closely aligned with what the customer needs.

Of course, the gap will begin to grow again immediately as development proceeds after each contact. The more frequent the contact points are, the easier it is to stay on track. As the progressively shrinking small gray triangles in Figure 2-1 illustrate, a series of such contact points will lead to a far smaller expectation gap at the end of the project and a solution that is much closer to the actual customer needs. This is why one of the guiding principles of agile development is to have ongoing conversations between developers and customers. That's an excellent principle for any project.

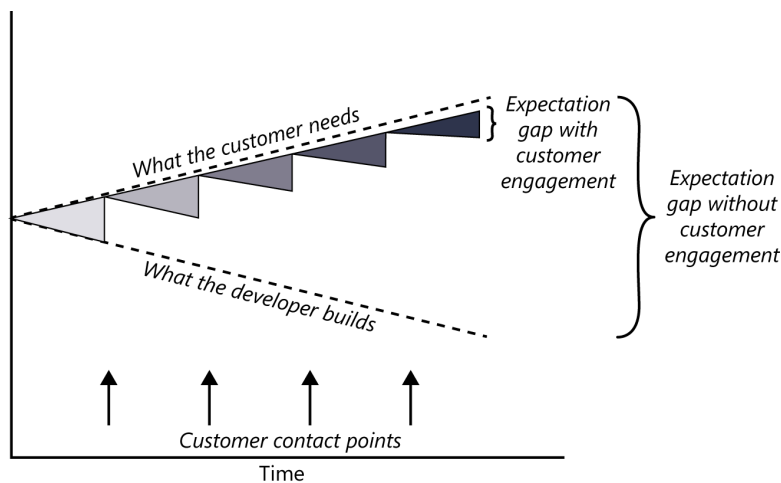


FIGURE 2-1 Frequent customer engagement reduces the expectation gap.

Who is the customer?

Before we can talk about customers, we need to discuss stakeholders. A *stakeholder* is a person, group, or organization that is actively involved in a project, is affected by its process or outcome, or can influence its process or outcome. Stakeholders can be internal or external to the project team and to the developing organization. Figure 2-2 identifies many of the potential stakeholders in these categories. Not all of these will apply to every project or situation, of course.

Stakeholder analysis is an important part of requirements development (Smith 2000; Wiegers 2007; IIBA 2009). When searching for potential stakeholders for a particular project, cast a wide net to avoid overlooking some important community. Then you can focus this candidate stakeholder list down to the core set whose input you really need, to make sure you understand all of the project's requirements and constraints so your team can deliver the right solution.

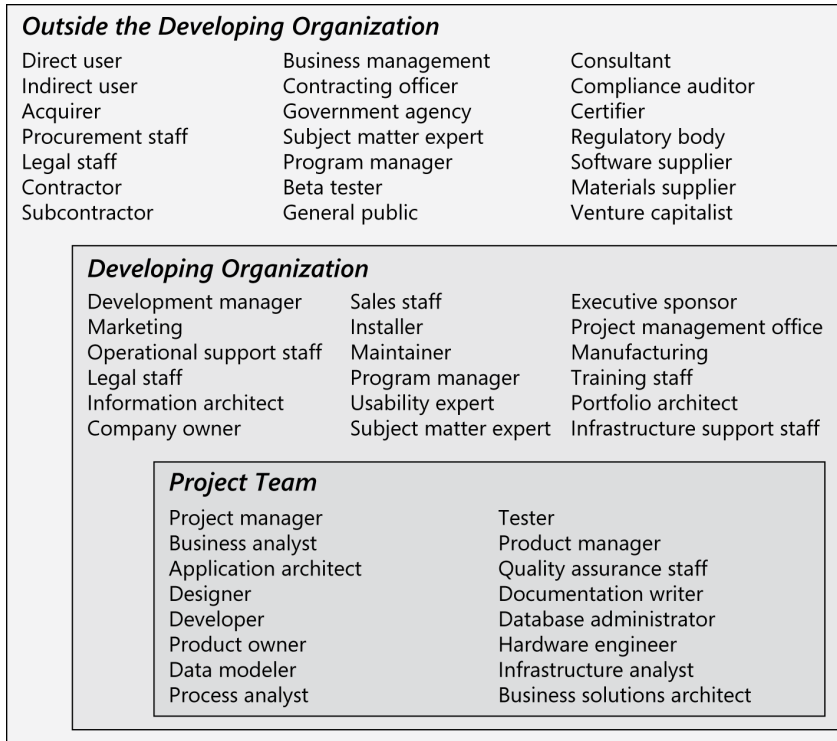


FIGURE 2-2 Potential stakeholders within the project team, within the developing organization, and outside the organization.

Customers are a subset of stakeholders. A *customer* is an individual or organization that derives either direct or indirect benefit from a product. Software customers could request, pay for, select, specify, use, or receive the output generated by a software product. The customers shown in Figure 2-2 include the direct user, indirect user, executive sponsor, procurement staff, and acquirer. Some stakeholders are not customers, such as legal staff, compliance auditors, suppliers, contractors, and venture capitalists. Gerhard, the manager we met earlier, represents an executive sponsor who is paying for the project. Customers like Gerhard provide the business requirements, which establish the guiding framework for the project and the business rationale for launching it. As discussed in Chapter 5, “Establishing the business requirements,” business requirements describe the business objectives that the customer, company, or other stakeholders want to achieve. All other product requirements need to align with achieving those desired business outcomes.

User requirements should come from people who will actually use the product, either directly or indirectly. These users (often called *end users*) are a subset of customers. Direct users will operate the product hands-on. Indirect users might receive outputs from the system without touching it themselves, such as a warehouse manager who receives an automatic report of daily warehouse activities by email. Users can describe the tasks they need to perform with the product, the outputs they need, and the quality characteristics they expect the product to exhibit.



The case of the missing stakeholder

I know of a project that was almost finished with requirements elicitation when, while reviewing a process flow, the business analyst (BA) asked the stakeholder, “Are you sure we have the tax calculation steps correct in this flow?” The stakeholder replied, “Oh, I don’t know. I don’t own tax. That’s the tax department.” The team hadn’t talked to anyone in the tax department over the course of working on the project for months. They had no idea that there even *was* a tax department. As soon as the BAs did meet with the tax department, they found a long list of missed requirements around the legal implications of how tax-related functions were implemented. The project was delayed several months as a result. Using an organization chart to search for all stakeholders who will be affected by a new system can avoid such unpleasantness.

Customers who provide the business requirements sometimes purport to speak for the actual users. They are often too far removed from the work to provide accurate user requirements, though. For corporate information systems, contract development, or custom application development, business requirements should come from the person who is ultimately accountable for the business value expected from the product. User requirements should come from people who will press the keys, touch the screen, or receive the outputs. If there is a serious disconnect between the acquiring customers who are paying for the project and the end users, major problems are guaranteed.

The situation is different for commercial software development, where the customer and the user often are the same person. Customer surrogates, such as marketing personnel or a product manager, typically attempt to determine what customers would find appealing. Even for commercial software, though, you should strive to engage end users in the process of developing user requirements, as Chapter 7, “Requirements elicitation,” describes. If you don’t, be prepared to read reviews pointing out product shortcomings that adequate user input could have avoided.

Conflicts can arise among project stakeholders. Business requirements sometimes reflect organizational strategies or budgetary constraints that aren’t apparent to users. Users who are upset about having a new information system forced on them by management might not want to work with the software developers, viewing them as the harbingers of an undesired future. Such folks are sometimes called “loser groups” (Gause and Weinberg 1989). To manage such potential conflicts, try communication strategies about project objectives and constraints that can build buy-in and avoid debates and hard feelings.

The customer-development partnership

An excellent software product results from a well-executed design based on excellent requirements. Excellent requirements result from effective collaboration between developers and customers (in particular, actual users)—a partnership. A collaborative effort can work only when all parties involved know what they need to be successful and when they understand and respect what their

collaborators need to be successful. As project pressures rise, it's easy to forget that all stakeholders share a common objective: to build a product that provides adequate business value and rewards to all stakeholders. The business analyst typically is the point person who has to forge this collaborative partnership.

The Requirements Bill of Rights for Software Customers in Table 2-1 lists 10 expectations that customers can legitimately hold regarding their interactions with BAs and developers during the project's requirements engineering activities. Each of these rights implies a corresponding responsibility on the part of the BAs or software developers. The word "you" in the rights and responsibilities refers to a customer for a software development project.

Because the flip side of a right is a responsibility, Table 2-2 lists 10 responsibilities that the customer has to BAs and developers during the requirements process. You might prefer to view these as a developer's bill of rights. If these lists aren't exactly right for your organization, modify them to suit the local culture.

TABLE 2-1 Requirements Bill of Rights for Software Customers

You have the right to
1. Expect BAs to speak your language.
2. Expect BAs to learn about your business and your objectives.
3. Expect BAs to record requirements in an appropriate form.
4. Receive explanations of requirements practices and deliverables.
5. Change your requirements.
6. Expect an environment of mutual respect.
7. Hear ideas and alternatives for your requirements and for their solution.
8. Describe characteristics that will make the product easy to use.
9. Hear about ways to adjust requirements to accelerate development through reuse.
10. Receive a system that meets your functional needs and quality expectations.

TABLE 2-2 Requirements Bill of Responsibilities for Software Customers

You have the responsibility to
1. Educate BAs and developers about your business.
2. Dedicate the time that it takes to provide and clarify requirements.
3. Be specific and precise when providing input about requirements.
4. Make timely decisions about requirements when asked.
5. Respect a developer's assessment of the cost and feasibility of requirements.
6. Set realistic requirement priorities in collaboration with developers.
7. Review requirements and evaluate prototypes.
8. Establish acceptance criteria.
9. Promptly communicate changes to the requirements.
10. Respect the requirements development process.

These rights and responsibilities apply to actual customers when the software is being developed for internal corporate use, under contract, or for a known set of major customers. For mass-market product development, the rights and responsibilities are more applicable to customer surrogates such as the product manager.

As part of project planning, the key customer and development stakeholders should review these two lists and negotiate to reach a meeting of the minds. Make sure the participants in requirements development understand and accept their responsibilities. This understanding can reduce friction later, when one party expects something that the other is not willing or able to provide.

Trap Don't assume that the project participants instinctively know how to collaborate on requirements development. Take the time to discuss how those involved can work together most effectively. It's a good idea to write down how you decide to approach and manage requirements issues on the project. This will serve as a valuable communication tool throughout the project.

Requirements Bill of Rights for Software Customers

Following are 10 rights that customers can expect when it comes to requirements issues.

Right #1: To expect BAs to speak your language

Requirements discussions should center on your business needs and tasks, using business vocabulary. Consider conveying business terminology to the BAs with a glossary of terms. You shouldn't have to wade through technical jargon when talking with BAs.

Right #2: To expect BAs to learn about your business and your objectives

By interacting with you to elicit requirements, the BAs can better understand your business tasks and how the system fits into your world. This will help developers create a solution that meets your needs. Invite BAs and developers to observe what you and your colleagues do on the job. If the new system is replacing an existing one, the BAs should use the current system as you use it. This will show them how it fits into your workflow and where it can be improved. Don't just assume that the BA will already know all about your business operations and terminology (see Responsibility #1).

Right #3: To expect BAs to record requirements in an appropriate form

The BA will sort through all the information that stakeholders provide and ask follow-up questions to distinguish user requirements from business rules, functional requirements, quality goals, and other items. The ultimate deliverable from this analysis is a refined set of requirements stored in some appropriate form, such as a software requirements specification document or a requirements management tool. This set of requirements constitutes the agreement among the stakeholders about

the functions, qualities, and constraints of the product to be built. Requirements should be written and organized in a way that you find easy to understand. Your review of these specifications and other requirements representations, such as visual analysis models, helps to ensure that they accurately represent your needs.

Right #4: To receive explanations of requirements practices and deliverables

Various practices can make requirements development and management both effective and efficient, and requirements knowledge can be represented in a variety of forms. The BA should explain the practices he's recommending and explain what information goes into each deliverable. For instance, the BA might create some diagrams to complement textual requirements. These diagrams might be unfamiliar to you, and they can be complex, but the notations shouldn't be difficult to understand. The BA should explain the purpose of each diagram, what the symbols mean, and how to examine the diagram for errors. If the BA doesn't offer such explanations, feel free to ask for them.

Right #5: To change your requirements

It's not realistic for BAs or developers to expect you to think of all your requirements up front or to expect those requirements to remain static throughout the development cycle. You have the right to make changes in the requirements as the business evolves, as the team gathers more input from stakeholders, or as you think more carefully about what you need. However, change always has a price. Sometimes adding a new function demands trade-offs with other functions or with the project's schedule or budget. An important part of the BA's responsibility is to assess, manage, and communicate change impacts. Work with the BA on your project to agree on a simple but effective process for handling changes.

Right #6: To expect an environment of mutual respect

The relationship between customers and developers sometimes becomes adversarial. Requirements discussions can be frustrating if the participants don't understand each other. Working together can open the eyes of the participants to the problems each group faces. Customers who participate in requirements development have the right to expect BAs and developers to treat them with respect and to appreciate the time they are investing in the project's success. Similarly, customers should demonstrate respect for the development team members as everyone collaborates toward their mutual objective of a successful project. Everyone's on the same side here.

Right #7: To hear ideas and alternatives for your requirements and for their solution

Let the BA know about ways that your existing systems don't fit well with your business processes to make sure that a new system doesn't automate ineffective or obsolete processes. That is, you want to avoid "paving the cow paths." A BA can often suggest improvements in your business processes. A creative BA also adds value by proposing new capabilities that customers haven't even envisioned.

Right #8: To describe characteristics that will make the product easy to use

You can expect BAs to ask you about characteristics of the software that go beyond your functional needs. These characteristics, or quality attributes, make the software easier or more pleasant to use, which lets users accomplish their tasks more efficiently. Users sometimes request that the product be *user-friendly* or *robust*, but such terms are too subjective to help the developers. Instead, the analyst should inquire about the specific characteristics that mean “user-friendly” or “robust” to you. Tell the BA about which aspects of your current applications seem “user-friendly” to you and which do not. If you don’t discuss these characteristics with the BA, you’ll be lucky if the product comes out as you hope.

Right #9: To hear about ways to adjust requirements to accelerate development through reuse

Requirements are often somewhat flexible. The BA might know of existing software components or requirements that come close to addressing some need you described. In such a case, the BA should suggest ways of modifying your requirements or avoiding unnecessary customizations so developers can reuse those components. Adjusting your requirements when sensible reuse opportunities are available saves time and money. Some requirements flexibility is essential if you want to incorporate commercial off-the-shelf (COTS) packages into the product, because they will rarely have precisely the characteristics you want.

Right #10: To receive a system that meets your functional needs and quality expectations

This is the ultimate customer right, *but* it can happen only if you clearly communicate all the information that will let developers build the right product, if developers communicate options and constraints to you, and if the parties reach agreement. Be sure to state all your assumptions and expectations; otherwise, the developers likely can’t address them properly. Customers sometimes don’t articulate points that they believe are common knowledge. However, validating a shared understanding across the project team is just as important as expressing something new.

Requirements Bill of Responsibilities for Software Customers

Because the counterpart to a right is a responsibility, following are 10 responsibilities that customer representatives have when it comes to defining and managing the requirements for their projects.

Responsibility #1: To educate BAs and developers about your business

The development team depends on you to educate them about your business concepts and to define business jargon. The intent is not to transform BAs into business experts but to help them understand your problems and objectives. BAs aren’t likely to be aware of knowledge that you and your peers take for granted.

Responsibility #2: To dedicate the time that it takes to provide and clarify requirements

Customers are busy people; those who are involved in requirements work are often among the busiest. Nonetheless, you have a responsibility to dedicate time to workshops, interviews, and other requirements elicitation and validation activities. Sometimes the BA might think she understands a point you made, only to realize later that she needs further clarification. Please be patient with this iterative approach to developing and refining the requirements; it's the nature of complex human communication and a key to software success. The total time required is less when there is focused effort for several hours than when the time is spent in bits and pieces strung out over weeks.

Responsibility #3: To be specific and precise when providing input about requirements

It's tempting to leave the requirements vague and fuzzy because pinning down details is tedious and time consuming (or because someone wants to evade being held accountable for his decisions). At some point, though, someone must resolve the ambiguities and imprecisions. You're the best person to make those decisions. Otherwise, you're relying on the BA or developers to guess correctly. It's fine to temporarily include *to be determined* (TBD) markers in the requirements to indicate that additional exploration or information is needed. Sometimes, though, TBD is used because a specific requirement is difficult to resolve and no one wants to tackle it. Try to clarify the intent of each requirement so that the BA can express it accurately. This is the best way to ensure that the product will meet your needs.

Responsibility #4: To make timely decisions about requirements when asked

Just as a contractor does while building your fabulous dream home, the BA will ask you to make many decisions. These include resolving conflicting requests received from multiple customers, choosing between incompatible quality attributes, and evaluating the accuracy of information. Customers who are authorized to make such decisions must do so promptly when asked. Developers often can't proceed with confidence until you render your decision, so time spent waiting for an answer can delay progress. When the demands for your time start to feel onerous, remember that the system is being built for you. Business analysts are often skilled at helping people think through making decisions, so ask for their help if you get stuck.

Responsibility #5: To respect a developer's assessment of the cost and feasibility of requirements

All software functions have a cost. Developers are in the best position to estimate those costs. Some features might not be technically feasible or might be surprisingly expensive to implement. Certain requirements might demand unattainable performance in the operating environment or require access to data that isn't available to the system. The developer can be the bearer of bad news about feasibility or cost. You should respect that judgment, even if it means you might not get something you asked for in exactly the form you envisioned. Sometimes, you can rewrite requirements in a way that makes them attainable or cheaper. For example, asking for an action to take place "instantaneously" isn't feasible, but a more precise timing requirement ("within 50 milliseconds") might be achievable.

Responsibility #6: To set realistic requirement priorities in collaboration with developers

Few projects have the time and resources to implement every bit of functionality all customers want. Determining which capabilities are essential, which are useful, and which the customers can live without is an important part of requirements analysis. You have a lead role in setting requirement priorities. Developers can provide information about the cost and risk of each requirement or user story to help determine final priorities. When you establish realistic priorities, you help the developers deliver the maximum value at the lowest cost and at the right time. Collaborative prioritization is key for agile projects, so the developers can begin delivering useful software as quickly as possible.

Respect the development team's judgment as to how much of the requested functionality they can complete within the available time and resource constraints. If everything you want doesn't fit in the project box, the decision makers will have to reduce project scope based on priorities, extend the schedule, or provide additional funds or people. Simply declaring every requirement as high priority is neither realistic nor collaborative.

Responsibility #7: To review requirements and evaluate prototypes

As you'll see in Chapter 17, "Validating the requirements," peer reviews of requirements are among the most powerful software quality activities available. Having customers participate in reviews is a key way to evaluate whether the requirements demonstrate the desired characteristics of being complete, correct, and necessary. A review is also an opportunity for customer representatives to assess how well the BA's work is meeting the project's needs. Busy customers often are reluctant to devote time to a requirements review, but it's well worth their time. The BA should make requirements available to you for review in manageable chunks throughout the requirements elicitation process, not in a massive tome dumped on your desk when the requirements are "done."

It's hard to develop a good mental picture of how software will work from written requirements alone. To better understand your needs and explore the best ways to satisfy them, BAs or developers sometimes build prototypes of the intended product. Your feedback on these preliminary, partial, or exploratory implementations provides valuable information to the developers.

Responsibility #8: To establish acceptance criteria

How do developers know when they're done? How can they tell if the software they built will meet the expectations of the various customer communities? As a customer, one of your responsibilities is to establish acceptance criteria, predefined conditions that the product must satisfy to be judged acceptable. Such criteria include acceptance tests, which assess whether the product lets users perform certain of their important business operations correctly. Other acceptance criteria might address the estimated remaining defect levels, the performance of certain actions in the operating environment, or the ability to satisfy external certification requirements. Agile projects rely heavily on acceptance tests, instead of written requirements, to flesh out the details of user stories. Testers can judge whether a specified requirement was implemented correctly, but they don't always know exactly what *you* will consider an acceptable outcome.

Responsibility #9: To promptly communicate changes to the requirements

Continually changing requirements pose a serious risk to the development team's ability to deliver a high-quality product on schedule. Change is inevitable and often valuable, but the later in development a change is introduced, the greater its impact. Notify the BA as soon as you learn that you need to change a requirement. To minimize the negative impact of changes, follow the project's defined change control process. This ensures that requested changes are not lost, the impact of each change is analyzed, and all proposed changes are considered in a consistent way. As a result, the business stakeholders can make sound business decisions to incorporate appropriate changes at the right stage of the project.

Responsibility #10: To respect the requirements development process

Eliciting and specifying requirements are among the greatest challenges in software development. There's a rationale behind the BA's approach to requirements development. Although you might become frustrated, the time spent understanding requirements is an excellent investment. The process will be less painful if you respect the techniques the BAs use. Feel free to ask BAs to explain why they're requesting certain information or asking you to participate in some requirements-related activity. A mutual understanding of, and respect for, each other's approaches and needs goes a long way toward establishing an effective—perhaps even enjoyable—collaboration.

Creating a culture that respects requirements



The leader of a corporate requirements organization once posed a problem: "I'm experiencing issues in gaining agreement from some of our developers to participate in requirements development," she said. "How can I help them understand the value of their participation?" In another organization, a BA experienced a clash between developers seeking detailed input for an accounting system and an IT manager who simply wanted to brainstorm requirements without using any specific elicitation techniques. "Do readers of your book risk cultural conflict?" this BA asked me.

These questions exemplify the challenges that can arise when trying to engage BAs, developers, and customers in a collaborative requirements partnership. You'd think it would be obvious to a user that providing requirements input makes it more likely that he'll get what he needs. Developers ought to recognize that participating in the process will make their lives easier than being hit on the head by whatever requirements document flies over the proverbial wall. Obviously, not everyone is as excited about requirements as you are; if they were, they'd probably all become business analysts!

Culture clashes frequently arise when teams are working on requirements. There are those who recognize the many risks associated with trying to develop software based on minimal or telepathically communicated requirements. Then there are those who think requirements are unnecessary. It can be tough to gain business-side cooperation on projects like legacy-system replacement if users see this as unrelated to their own business problems and not worth their time. Understanding why people resist participating in requirements development is the first step to being able to address it.

It's possible that the resisters haven't been exposed to solid requirements practices. Or they might have suffered from poor implementation of requirements processes, perhaps working on a project that produced a large, incomplete, and ignored requirements specification. That would leave a bad taste in anyone's mouth. Perhaps the resisters don't understand and appreciate the value of those practices when performed effectively. They might not realize the price they have paid for having worked in a casual and unstructured environment in the past. That price mostly shows up as unexpected rework that leads to late deliveries and poor software. Such rework is buried in the daily activities of the project participants, so they don't recognize it as a serious inefficiency.

If you're trying to get developers, managers, and customers on board, make sure everyone understands the past pain the organization and its customers have experienced because of requirements problems. Find specific examples to demonstrate the impact in case individuals haven't felt the pain themselves. Express the cost in units that are meaningful to the organization, be it dollars, time, customer dissatisfaction, or lost business opportunities. Development managers aren't always aware of how badly requirements shortcomings hurt their teams' productivity. So show them how poor requirements slow down design and lead to excessive—and expensive—course corrections.

Developers are stakeholders in the project, but sometimes their input isn't solicited and they become the "victims" of the requirements that are thrust upon them. Therefore, they benefit from providing input that will make the requirements documentation as useful and meaningful as possible. I like to have developers review requirements as they are evolving. That way they know what's coming and can spot areas that need more clarity. You also need developer input when specifying internal quality attributes that aren't visible to users. Developers can offer suggestions no one else might have thought about: easier ways to do certain things; functionality that would be very time-consuming to implement; unnecessary imposed design constraints; missing requirements, such as how exceptions should be handled; and creative opportunities to take advantage of technologies.

Quality assurance staff and testers are also valuable contributors to excellent requirements. Instead of waiting until later in the project, engage these sharp-eyed people in the iterative review of requirements early on. They're likely to find many ambiguities, conflicts, and concerns with the requirements as they are developing their test cases and scenarios from the requirements. Testers can also provide input on specifying verifiable quality attribute requirements.

Resistance to process or culture change can indicate fear, uncertainty, or lack of knowledge. If you can discern the source of the resistance, you can confront it with reassurance, clarification, and education. Show people how their participation not only is in their personal best interest but also will lead to collectively better results.

The organization's leadership must understand the need for the organization to have effective business analysis and requirements engineering capabilities as strategic core competencies. Though project-specific and localized grassroots efforts are important, without management commitment, the improvements and benefits likely won't be sustained after the project ends or following a reorganization.

Identifying decision makers



There can be hundreds of decisions to make on software projects; often, they are on the critical path to being able to move ahead. You might need to resolve some conflict, accept (or reject) a proposed change, or approve a set of requirements for a specific release. Early in your project, determine who the requirements decision makers will be and how they will make decisions. My friend Chris, a seasoned project manager, pointed out, “I have found that there is usually one primary decision maker on a project, oftentimes the key sponsor within the organization. I don’t rest until I have identified that person, and then I make sure he is always aware of the project’s progress.” There’s no single correct answer as to who should make key decisions. A small group representing key areas—such as management, customers, business analysis, development, and marketing—generally works best. Chapter 28, “Change happens,” describes the change control board, which serves as the decision makers for proposed requirement changes.

The decision-making group needs to identify its *decision leader* and to select a *decision rule*, which describes how they will arrive at their decisions. There are numerous decision rules to choose from, including the following (Gottesdiener 2001):

- The decision leader makes the choice, either with or without discussion with others.
- The group votes and the majority rules.
- The group votes, but the result must be unanimous to approve the decision.
- The group discusses and negotiates to reach a consensus. Everyone can live with the decision and commits to supporting it.
- The decision leader delegates authority for making the decision to one individual.
- The group reaches a decision, but some individual has veto authority over that decision.

There is no globally correct or appropriate decision rule. A single decision rule won’t work in every situation, so the group must establish guidelines so they know when to vote, when to reach consensus, when to delegate, and so on. The people who will be making requirements-related decisions on each of your projects should choose a decision rule before they confront their first significant decision.

Reaching agreement on requirements

Reaching agreement on the requirements for the product to be built, or for a specific portion of it, is at the core of the customer-developer partnership. Multiple parties are involved in this agreement:

- Customers agree that the requirements address their needs.
- Developers agree that they understand the requirements and that they are feasible.
- Testers agree that the requirements are verifiable.
- Management agrees that the requirements will achieve their business objectives.

Many organizations use the act of “signing off” (why not “signing on”?) on the requirements as the mark of stakeholder approval. All participants in the requirements approval process should know exactly what sign-off means or problems could ensue. One such problem is the customer representative or manager who regards signing off on the requirements as a meaningless ritual: “I was handed a piece of paper with my name on it, so I signed on the line above my name because otherwise the developers wouldn’t start coding.” This can lead to future problems when that individual wants to change the requirements or when he’s surprised by what is delivered: “Sure, I signed off on the requirements, but I didn’t have time to read them all. I trusted you guys—you let me down!”

Equally problematic is the development manager who views sign-off as a way to freeze the requirements. Whenever a change request comes along he can protest, “But you signed off on these requirements, so that’s what we’re building. If you wanted something else, you should have said so.”

Both of these attitudes ignore the reality that it’s impossible to know all the requirements early in the project and that requirements will undoubtedly change over time. Approving a set of requirements is an appropriate action that brings closure to some stage of requirements development. However, the participants have to agree on precisely what they’re saying with their signatures.



Important Don’t use sign-off as a weapon. Treat it as a milestone, with a clear, shared understanding of the activities that lead to sign-off and its implications for future changes. If the decision makers don’t need to read every word of the requirements, select a communication technique—such as a slide presentation—that summarizes the essential elements and facilitates reaching agreement quickly.

The requirements baseline

More important than the sign-off ritual is the concept of establishing a *baseline* of the requirements agreement, a snapshot of it at a point in time (Wiegers 2006). A requirements baseline is a set of requirements that has been reviewed and agreed upon and serves as the basis for further development. Whether your team uses a formal sign-off process or some other means of reaching agreement on requirements, the subtext of that agreement should read something like this:

“I agree that this set of requirements represents our best understanding of the requirements for the next portion of this project and that the solution described will meet our needs as we understand them today. I agree to make future changes in this baseline through the project’s defined change process. I realize that changes might require us to renegotiate cost, resource, and schedule commitments.”

Some organizations put text like this right on the signature page, so the requirement approvers know exactly what sign-off means in their world.

A shared understanding along these lines helps reduce the friction that can arise as requirements oversights are revealed or marketplace and business demands evolve over the course of the project. A meaningful baselining process gives all the major stakeholders confidence in the following ways:

- Customer management or marketing is confident that the project scope won't explode out of control, because customers manage the scope change decisions.
- User representatives have confidence that the development team will work with them to deliver the right solution, even if they didn't think of every requirement before construction began.
- Development management has confidence because the development team has a business partner who will keep the project focused on achieving its objectives and will work with development to balance schedule, cost, functionality, and quality.
- Business analysts and project managers are confident that they can manage changes to the project in a way that will keep chaos to a minimum.
- Quality assurance and test teams can confidently develop their test scripts and be fully prepared for their project activities.

After the decision makers define a baseline, the BA should place the requirements under change control. This allows the team to modify scope when necessary in a controlled way that includes analyzing the impact of each proposed change on the schedule and other success factors. Sealing the initial requirements development activities with an explicit agreement helps forge a collaborative customer-development partnership on the way to project success.

What if you don't reach agreement?

It can be hard to achieve sign-off from all the relevant stakeholders. Barriers include logistics, busy schedules, and people who are reluctant to commit and be held accountable later. If stakeholders are afraid they won't be able to make changes after they approve the requirements, they might drag their feet on the approval. This contributes to the dreaded trap of analysis paralysis. Many teams have tried sending out an email message that says, "If you don't reply by next Friday with your changes and/or sign-off, I'm going to assume you are agreeing to these requirements." That's one option, but really it equates to *not* reaching agreement. It also risks straining the relationship with those stakeholders for whom you've just assumed a tacit approval. Try to understand why they didn't feel comfortable with a sign-off and address that directly.

In such a situation, you're better off moving forward—cautiously—with the assumption that you don't have approval from the recalcitrant stakeholders. Document the fact that certain stakeholders didn't sign off on the requirements in your risk list, along with the likely impact of some of the requirements being missing or wrong. Follow up with these people as part of risk management. In a positive manner, mention that you recognize that they have not yet approved the requirements but that the project is still moving forward with those requirements as a baseline so as to not impede progress. Let them know that, if they want to change things, there's a process in place to do that. Basically, you're acting as though the stakeholder did indeed agree to the requirements, but you're managing the communications closely.

Agreeing on requirements on agile projects

Agile projects do not include a formal sign-off action. Agile projects generally maintain requirements in the form of user stories in a product backlog. The product owner and the team reach agreement on what stories will be developed in the next iteration in a planning session. The set of stories is chosen based on their priority and the team's velocity (productivity). After that set has been established and agreed to, the stories contained in the iteration are frozen. Requested changes that come in are considered for future iterations. There's no attempt on an agile project to achieve stakeholder approval on the full scope of requirements for the project up front, however. In agile projects the full set of functionality is identified over time, although the vision and other business requirements do need to be established at the outset. Chapter 20, "Agile projects," discusses how requirements are handled on agile projects.



I once worked with a client who requested sign-off on requirements even though they were following an agile development life cycle. The team had to be creative with how to do this in a context that doesn't traditionally involve sign-offs. The BA team had worked closely with the users to elicit and review requirements in the form of user stories and other models such as process flows and state tables. We asked the users to "sign off" that, at that moment in time, there were no major requirements missing *that they knew about*, and there were no major issues with what we'd written down *that they knew about*. Because users did participate in the requirements activities, development would not be working on a solution that would be far off base. But this notion of "sign-off" also keeps open the right of the users to realize later on that they need something new or got something wrong.

In contrast to the historical notion of sign-off as meaning "approve and freeze all the requirements up front," this approach doesn't force anyone into a corner where he feels like he's signing away his life over a massive requirements document that he barely understands. Nor are customers forced to agree that the requirements are close to perfect and that everything was addressed the first time around. This version of sign-off allows the spirit of agile methods to prevail. As with the sign-off process described earlier, the essence is to reach agreement on a specific body of requirements—a baseline—to be implemented in the next construction cycle, with a clear, shared understanding of what that agreement really means.

Commonly on agile projects, the product owner publicly accepts or rejects the requirements for an iteration, which consist of a set of stories and their accompanying acceptance criteria and acceptance tests. The ultimate "sign-off" is acceptance of the working, tested software delivered from the iteration.

As consultant Nanette Brown put it, "Even in an agile environment the concept of sign-off can fill a valid function. Agile tells us to 'embrace change,' but the concept of change only exists with respect to a reference point. Even within a team where there is close communication, people can have different interpretations of current plans and status. One person's 'change' can be what another person thought was already agreed to. However, if you position a sign-off as a lightweight ceremony acknowledging that 'We are Here' I think it's fine. Just because 'We are Here' today doesn't mean we can't be somewhere else tomorrow, but at least it ensures a common understanding and point of reference."



Next steps

- Identify the customers, including end users, who are responsible for providing business and user requirements on your project. Which items from the Bill of Rights and the Bill of Responsibilities do these customers currently accept and practice? Which do they not?
- Discuss the Bill of Rights with your key customers to learn whether they feel they aren't receiving any of their rights. Discuss the Bill of Responsibilities to reach agreement as to which responsibilities they accept. Modify the Bill of Rights and the Bill of Responsibilities as appropriate so that all parties agree on how they will work together. Monitor whether the stakeholders are maintaining a balance between rights and responsibilities.
- If you're a customer participating in a software project and you don't feel that your requirements rights are being adequately respected, discuss the Bill of Rights with the project manager or the BA. Offer to do your part to satisfy the Bill of Responsibilities as you strive to build a more collaborative working relationship.
- If your organization uses a formal sign-off process, think about what it really means today. Work with development and customer (or marketing) management to reach agreement on what sign-off really ought to mean for your requirements approval process.
- Identify one example from a current or past project of not having the necessary level of customer participation. Consider what the impact of that was. See if you can quantify the risk in terms of number of late requirements changes, time spent fixing the product after delivery, or business opportunities missed. Use that experience in the future as a story to learn from and to convince others of why customer engagement is so vital.